

[Ville Voutilainen](#)

[Jonathan Wakely](#)

[John Spicer](#)

[Stephan T. Lavavej](#)

2025-11-06

Standard library hardening should not use the 'observe' semantic

Revision history

The first revision of this paper presented information on some problems of using C++26 Contracts for standard library hardening. The second revision no longer includes most of that, we now just propose a change to ensure that undefined behaviour is not allowed after a hardened precondition is violated.

Abstract

The C++26 IS draft specifies that hardened preconditions may be, in a hardened implementation, evaluated with a non-terminating semantic. This can result in violations of hardened preconditions being undefined behaviour, rather than guaranteed to be diagnosed, which defeats the purpose of using a hardened implementation.

A hardened implementation that uses 'observe' is not hardened

The specification says, in [structure.specifications]/3.5.1, with the problematic part bolded,

When invoking the function in a hardened implementation, prior to any other observable side effects of the function, one or more contract assertions whose predicates are as described in the hardened precondition are evaluated with a checking semantic (6.11.2). **If any of these assertions is evaluated with a non-terminating semantic and the contract-violation handler returns, the program has undefined behavior.**

That is completely unacceptable for a hardened implementation. A hardened implementation is a memory safety mechanism, and it's not memory-safe to run into the UB that results in calling a library function violating its preconditions.

While there are use cases for having observed hardened preconditions, for code that's common in the wild, doesn't actually run into abstract machine UB, and works fine, it doesn't make sense for that to be allowed by a "hardened implementation". Nothing prevents vendors from offering a "checked but not hardened" mode where the hardened preconditions are evaluated using the 'observe' semantic and invoking a custom violation handler. Users who want that can certainly still get that, if their implementation provides it. But we should not allow anybody to claim that such a mode is a conforming "hardened implementation".

If a piece of software has requirements that it be built using a hardened implementation (e.g. due to contractual obligations for a customer, or hypothetical laws that might be applied to some regulated industries) then presumably there is a desire and expectation that undefined behaviour resulting from violating hardened preconditions will be **prevented**, not just observed and logged.

Proposed change

We propose that in a hardened implementation, the assertions should be evaluated using a terminating semantic. This would no longer allow the 'observe' semantic in a hardened implementation (but checking those preconditions using 'observe' would still be allowed as a vendor extension in modes which are not called a "hardened implementation").

The proposed change does not guarantee that violating a hardened precondition in a program built with a hardened implementation will result in the program being contract-terminated. If a hardened implementation supports replacing the contract violation handler, the program can use a handler that throws, which will mean the program is not terminated. However, this still avoids the concerns presented above, because throwing will prevent the rest of the standard library function from being executed after the precondition violation was detected. There will be no undefined behaviour in the body of that function.

Allowing 'observe' for hardened preconditions was a mistake

The authors of the standard library hardening proposal consider the change proposed in this paper to be a bug fix. They agree with the position here that a hardened implementation should not allow undefined behaviour after a violation is detected.

The submitters of NB comments about standard library hardening confirmed that this change resolves their concerns.

Wording

In [structure.specifications]/3.1.5, edit as follows:

When invoking the function in a hardened implementation, prior to any other observable side effects of the function, ~~one or more~~ contract assertions whose predicates are as described in the hardened precondition are evaluated with a ~~checking~~terminating semantic (6.11.2). ~~If any of these assertions is evaluated with a non-terminating semantic and the contract violation handler returns, the program has undefined behavior.~~